

# 09 Flood Risk

EP

2026-08-25

This notebook runs a Random Forest, XGBoost, and SHAP for Species richness without the flood risk factors: Risk of Flooding from Rivers and Sea (RoFRS) and Risk of Flooding from Surface Water (RoFSW). This notebook intends to provide evidence for or against the use of these flood risk indicators in these models.

## Libraries

```
library(targets)
library(tidymodels)
library(ranger)
library(sf)
library(dplyr)
library(xgboost)
library(kernelshap)
library(shapviz)
library(stringr)
library(patchwork)

set.seed(399)
```

## Load Data

```
grid_model_all <- st_read("Data/grid_model.gpkg", quiet = TRUE)

df <- grid_model_all %>%
  sf::st_drop_geometry()

predictors <- c("dtm_elev", "avg_imperv", "avg_area_ha", "treeht", "weighted_precip", "weighted_temp", "ClimoTemp", "ClimoPrecip", "ClimoArea", "ClimoTreeht", "ClimoImperv", "ClimoElev")
```

## Setup

```
#Classification setup - median split
df <- df %>%
  mutate(
    richness_class = factor(
      if_else(sp_richness >= median(sp_richness, na.rm = TRUE), "high", "low"),
```

```

    levels = c("low", "high"))

#Folds & fold city names
folds <- group_vfold_cv(df, group = city)

fold_city_lookup <- folds %>%
  mutate(city = purrr::map_chr(
    splits, ~ as.character(unique(assessment(.x)$city))) %>%
    select(id, city)

```

## Random Forest

### Specification

```

#RF Specification
rf_spec_class <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 500) %>%
  set_engine("ranger", importance = "permutation",
    respect.unordered.factors = "order", probability = TRUE) %>%
  set_mode("classification")

#RF Recipe and Workflow
richness_recipe_class <- recipe(
  as.formula(paste("richness_class ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  step_naomit(all_predictors(), all_outcomes())

richness_wf_class <- workflow() %>%
  add_recipe(richness_recipe_class) %>%
  add_model(rf_spec_class)

rf_grid <- grid_regular(
  mtry(range = c(2, 8)),
  min_n(range = c(2, 15)),
  levels = 5) #Total of 25 combinations in the grid.

```

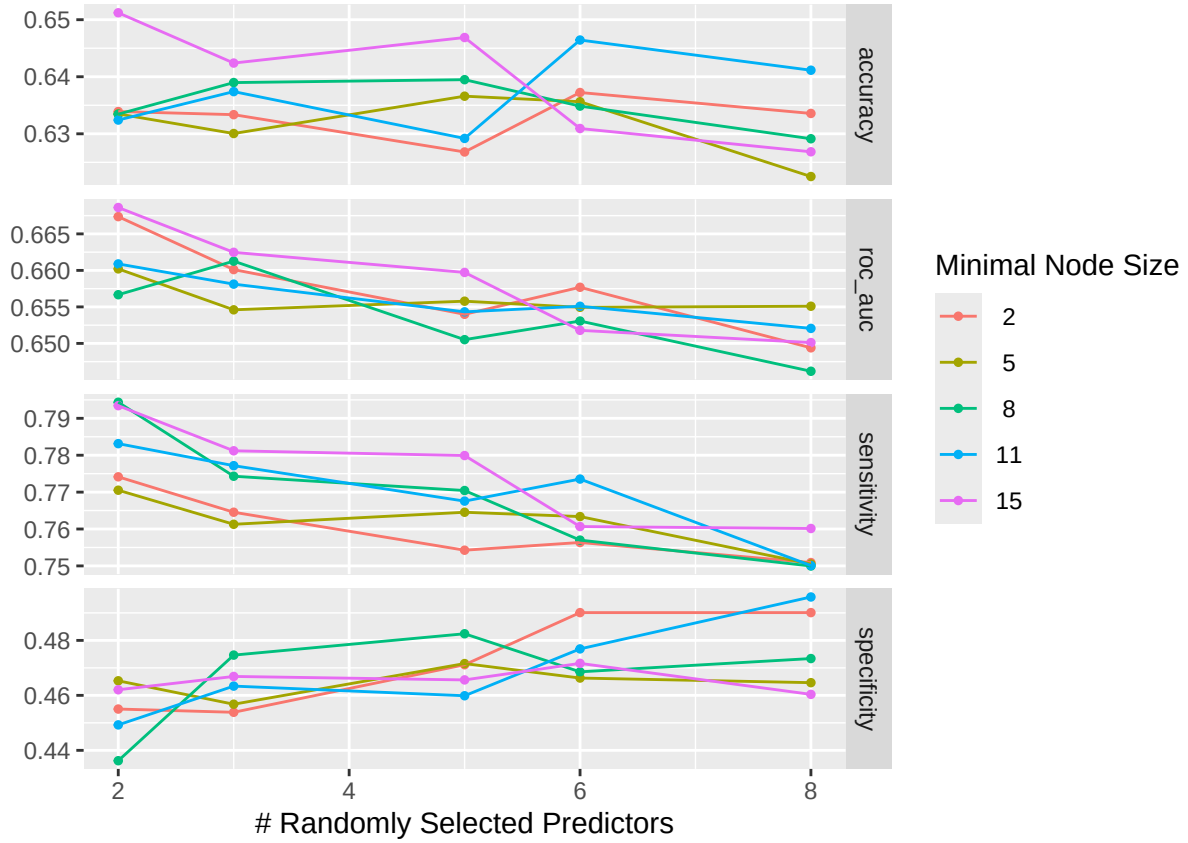
### Hyperparameter Tuning

```

# Use specification to run hypertuning
richness_res_class <- tune_grid(
  richness_wf_class,
  resamples = folds,
  grid      = rf_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

autoplot(richness_res_class)

```



### Select Best Hyperparameters

```
show_best(richness_res_class, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(
    caption = "Top 5 hyperparameter combinations:
    species richness class (ranked by ROC AUC)"
```

Table 1: Top 5 hyperparameter combinations: species richness class (ranked by ROC AUC)

| mtry | min_n | Mean ROC AUC | Std. error |
|------|-------|--------------|------------|
| 2    | 15    | 0.669        | 0.029      |
| 2    | 2     | 0.667        | 0.028      |
| 3    | 15    | 0.662        | 0.026      |
| 3    | 8     | 0.661        | 0.026      |
| 2    | 11    | 0.661        | 0.029      |

```
best_richness_class <- select_best(richness_res_class, metric = "roc_auc")
```

## Prediction Rate AUC

```
# Get overall prediction rate AUC for best model
prediction_rate_richness <- collect_metrics(richness_res_class) %>%
  filter(mtry == best_richness_class$mtry,
         min_n == best_richness_class$min_n,
         .metric == "roc_auc")

overall_richness <- prediction_rate_richness %>%
  pull(mean)

# Get pred rate per fold (i.e. by city)
prediction_rate_richness_byfold <- collect_metrics(
  richness_res_class, summarize = FALSE) %>%
  filter(mtry == best_richness_class$mtry,
         min_n == best_richness_class$min_n,
         .metric == "roc_auc")

# Get city names
city_richness <- prediction_rate_richness_byfold %>%
  left_join(fold_city_lookup, by="id") %>%
  select(city, .estimate) %>%
  rename(richness_auc = .estimate)

# Make overall a city for formatting
overall_auc <- tibble(city = "Overall",
                      richness_auc = overall_richness)

# Combine and Plot
classification_auc_table <- bind_rows(overall_auc, city_richness) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  rename("City (Held-Out)" = city,
         "Richness ROC AUC" = richness_auc)

classification_auc_table %>%
  knitr::kable(
    caption = "Overall and city-level cross-validated ROC AUC for species richness")
```

Table 2: Overall and city-level cross-validated ROC AUC for species richness

| City (Held-Out) | Richness ROC AUC |
|-----------------|------------------|
| Overall         | 0.669            |
| Nottingham      | 0.632            |
| Exeter          | 0.620            |
| Newcastle       | 0.634            |
| Bristol         | 0.779            |
| Gateshead       | 0.678            |

```
# Check for any folds where AUC couldn't be computed (single-class fold)
richness_na_folds <- prediction_rate_richness_byfold %>%
  filter(is.na(.estimate))

richness_na_folds
```

```
## # A tibble: 0 x 7
## # i 7 variables: id <chr>, mtry <int>, min_n <int>, .metric <chr>,
## #   .estimator <chr>, .estimate <dbl>, .config <chr>
```

## Final Model

```
final_richness_wf_class <- finalize_workflow(
  richness_wf_class, best_richness_class)
richness_fit_class <- fit(final_richness_wf_class, data = df)
```

## Success Rate AUC

```
richness_success <- augment(richness_fit_class, new_data = df)

success_rate_richness <- roc_auc(richness_success, truth = richness_class, .pred_high, event_level = "s
```

## Summary Table

```
tibble::tibble(
  Outcome = "Species richness",
  `Success rate AUC` = success_rate_richness$.estimate,
  `Prediction rate AUC` = prediction_rate_richness$mean,
  `Prediction rate SE` = prediction_rate_richness$std_err) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  knitr::kable(caption = "Random forest classification performance: success rate vs. prediction rate AU
```

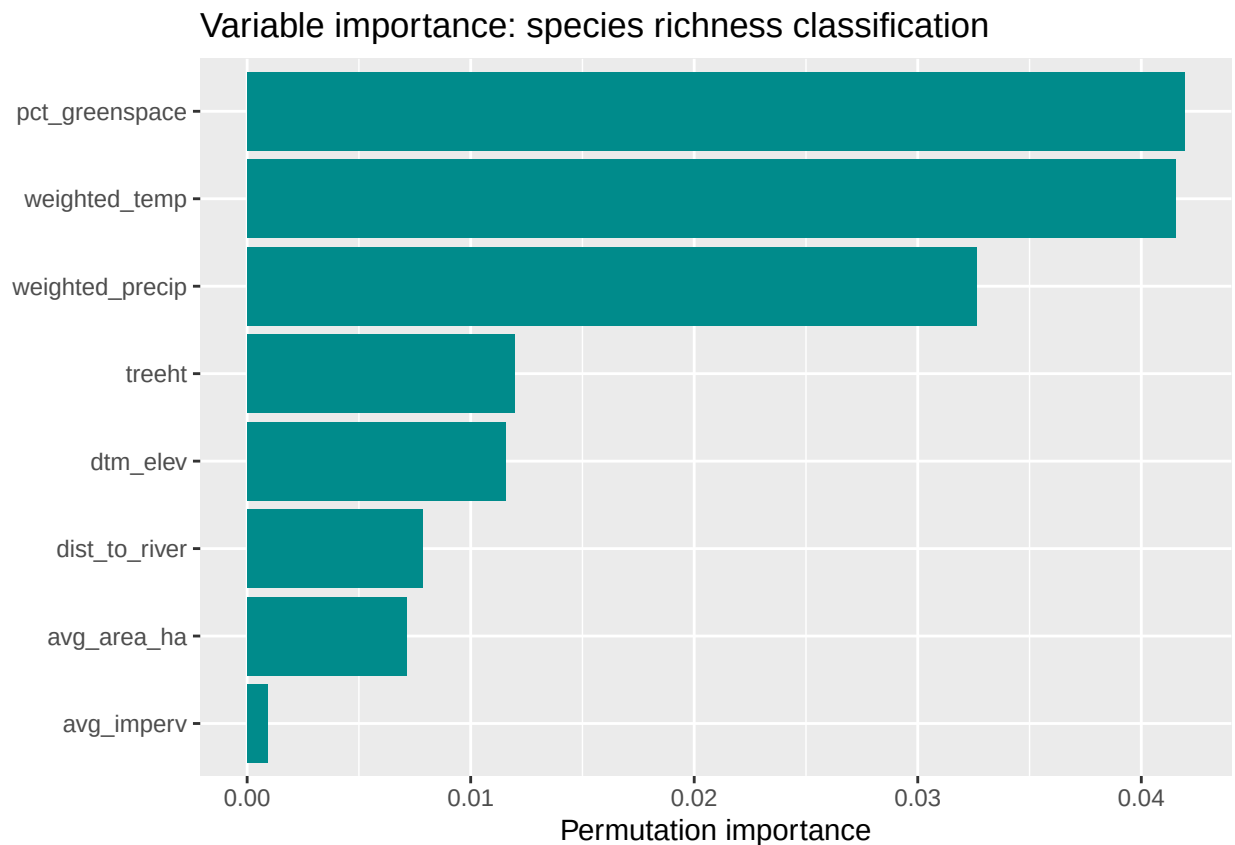
Table 3: Random forest classification performance: success rate  
vs. prediction rate AUC

| Outcome          | Success rate AUC | Prediction rate AUC | Prediction rate SE |
|------------------|------------------|---------------------|--------------------|
| Species richness | 0.994            | 0.669               | 0.029              |

## Variable Importance

```
# Calculate and plot variable importance using `ranger`
p1 <- richness_fit_class %>%
  extract_fit_engine() %>%
  ranger::importance() %>%
  sort(decreasing = TRUE) %>%
  as.data.frame() %>%
  tibble::rownames_to_column("variable") %>%
  rename(importance = 2) %>%
  ggplot(aes(x = reorder(variable, importance), y = importance)) +
  geom_col(fill = "cyan4") +
  coord_flip() +
  labs(title = "Variable importance: species richness classification",
       x = NULL, y = "Permutation importance")
```

p1



Compared to RF modelling with flood risk factors, this model shows a decrease in prediction rate AUC (0.688 to 0.669), increase in success rate AUC (0.945 to 0.994), and minimal change in standard error (0.027 to 0.029). Top three variable importance predictors remain the same: **pct\_greenpace**, **weighted\_temp**, and **weighted\_precip**. Whilst this increase (0.019) in predictive performance exists, it is small relative to the standard error making any conclusions uncertain.

# XGBoost

## Specification & workflow

```
#Specification
xgb_spec_class <- boost_tree(
  trees          = tune(),
  tree_depth     = tune(),
  learn_rate     = tune(),
  loss_reduction = tune(),
  mtry           = tune(),
  min_n          = tune(),
  sample_size    = tune() %>%
  set_engine("xgboost") %>%
  set_mode("classification")

#Recipes and workflow
richness_recipe_xgb <- recipe(
  as.formula(paste("richness_class ~", paste(predictors, collapse = " + "))),
  data = df) %>%
  step_unknown(all_nominal_predictors()) %>%
  step_naomit(all_numeric_predictors(), all_outcomes()) %>%
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors())

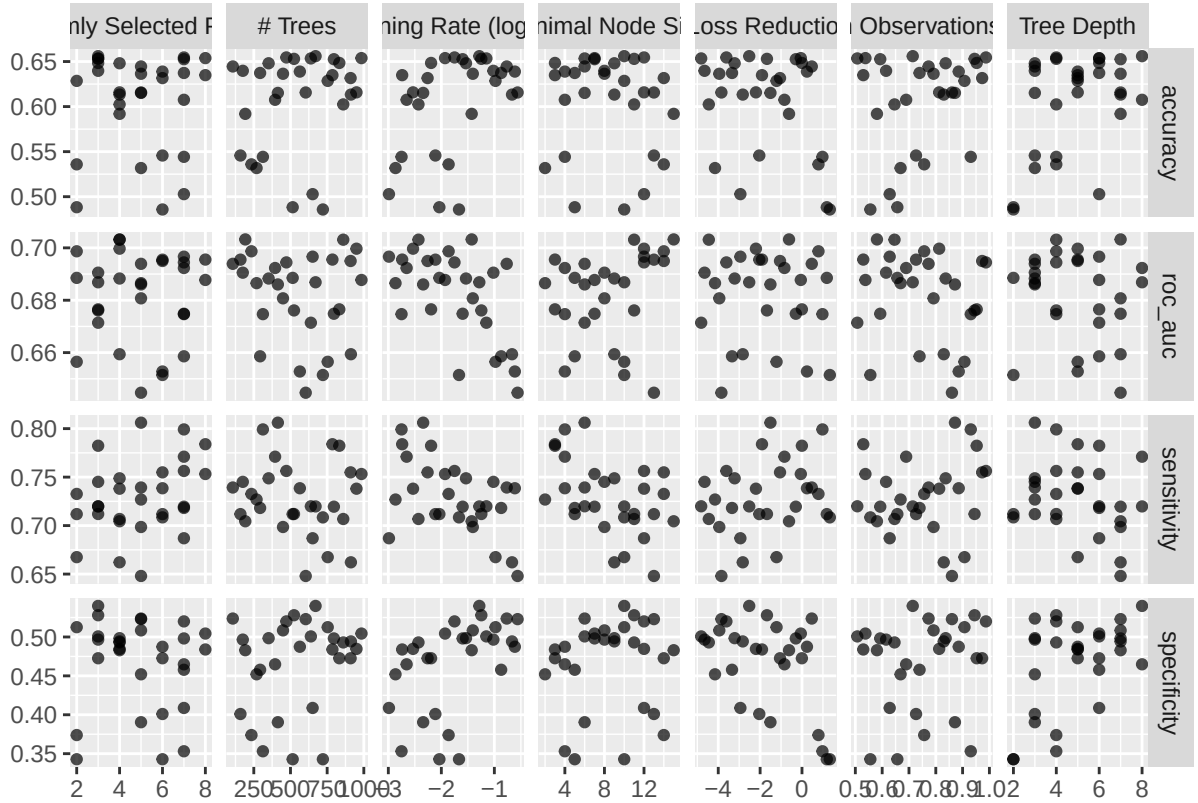
richness_wf_xgb <- workflow() %>%
  add_recipe(richness_recipe_xgb) %>%
  add_model(xgb_spec_class)
```

## Tuning

```
#Setup
xgb_grid <- grid_space_filling(
  trees(range = c(100, 1000)),
  tree_depth(range = c(2, 8)),
  learn_rate(range = c(-3, -0.5)), # log10 scale: 0.001 - 0.316
  loss_reduction(range = c(-5, 1.5)),
  finalize(mtry(range = c(2, 8)), df[, predictors]),
  min_n(range = c(2, 15)),
  sample_prop(range = c(0.5, 1.0)),
  size = 30,
  type = "latin_hypercube")

#Hyperparameter tuning
richness_res_xgb <- tune_grid(
  richness_wf_xgb,
  resamples = folds,
  grid      = xgb_grid,
  metrics   = metric_set(roc_auc, accuracy, sensitivity, specificity))

autoplot(richness_res_xgb)
```



## Select Best

```
show_best(richness_res_xgb, metric = "roc_auc", n = 5) %>%
  select(mtry, min_n, tree_depth, learn_rate, loss_reduction,
         sample_size, trees, mean, std_err) %>%
  mutate(across(c(mean, std_err), ~ round(.x, 3))) %>%
  rename("Mean ROC AUC" = mean, "Std. error" = std_err) %>%
  knitr::kable(caption = "Top 5 hyperparameter combinations:
                    species richness class (XGBoost, ranked by ROC AUC)")
```

Table 4: Top 5 hyperparameter combinations: species richness class  
(XGBoost, ranked by ROC AUC)

| mtry | min_n | tree_depth | learn_rate | loss_reduction | sample_size | trees | Mean ROC |            |
|------|-------|------------|------------|----------------|-------------|-------|----------|------------|
|      |       |            |            |                |             |       | AUC      | Std. error |
| 4    | 15    | 7          | 0.0373615  | 0.2405106      | 0.5806951   | 192   | 0.703    | 0.027      |
| 4    | 11    | 4          | 0.0037041  | 0.0000355      | 0.6460809   | 860   | 0.703    | 0.032      |
| 4    | 12    | 5          | 0.0029253  | 0.0063202      | 0.8126605   | 950   | 0.700    | 0.035      |
| 2    | 14    | 4          | 0.0137258  | 6.1288748      | 0.7567704   | 234   | 0.699    | 0.028      |
| 7    | 12    | 6          | 0.0010276  | 0.0011435      | 0.6282488   | 651   | 0.697    | 0.033      |



```
best_richness_xgb <- select_best(richness_res_xgb, metric = "roc_auc")
```

## Prediction rate AUC

```
# As before, collect overall and by city AUCs. Create table.
overall_richness_xgb <- collect_metrics(richness_res_xgb) %>%
  filter(mtry == best_richness_xgb$mtry, min_n == best_richness_xgb$min_n,
         tree_depth == best_richness_xgb$tree_depth,
         learn_rate == best_richness_xgb$learn_rate,
         .metric == "roc_auc") %>%
  pull(mean)

city_richness_xgb <- collect_metrics(richness_res_xgb, summarize = FALSE) %>%
  filter(mtry == best_richness_xgb$mtry, min_n == best_richness_xgb$min_n,
         tree_depth == best_richness_xgb$tree_depth,
         learn_rate == best_richness_xgb$learn_rate,
         .metric == "roc_auc") %>%
  left_join(fold_city_lookup, by = "id") %>%
  select(city, .estimate) %>%
  rename(richness_auc = .estimate)

overall_auc_xgb <- tibble(city = "Overall", richness_auc = overall_richness_xgb)

classification_auc_table_xgb <- bind_rows(overall_auc_xgb, city_richness_xgb) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
  rename("City (Held-Out)" = city,
         "Richness ROC AUC" = richness_auc)

classification_auc_table_xgb %>%
  knitr::kable(caption = "Overall and city-level cross-validated ROC AUC for species richness (XGBoost)")
```

Table 5: Overall and city-level cross-validated ROC AUC for species richness (XGBoost)

| City (Held-Out) | Richness ROC AUC |
|-----------------|------------------|
| Overall         | 0.703            |
| Nottingham      | 0.696            |
| Exeter          | 0.652            |
| Newcastle       | 0.651            |
| Bristol         | 0.796            |
| Gateshead       | 0.720            |

## Final Model

```
final_richness_wf_xgb <- finalize_workflow(richness_wf_xgb, best_richness_xgb)
richness_fit_xgb <- fit(final_richness_wf_xgb, data = df)
```

## Success rate AUC

```
richness_success_xgb <- augment(richness_fit_xgb, new_data = df)

success_rate_richness_xgb <- roc_auc(richness_success_xgb, truth = richness_class, .pred_high, event_level = "none")
```

## Summary Table

```
tibble::tibble(
  Outcome = "Species richness",
  `Success rate AUC` = success_rate_richness_xgb$.estimate,
  `Prediction rate AUC` = overall_richness_xgb) %>%
mutate(across(where(is.numeric), ~ round(.x, 3))) %>%
knitr::kable(caption = "XGBoost classification performance:
  success rate vs. prediction rate AUC")
```

Table 6: XGBoost classification performance: success rate vs. prediction rate AUC

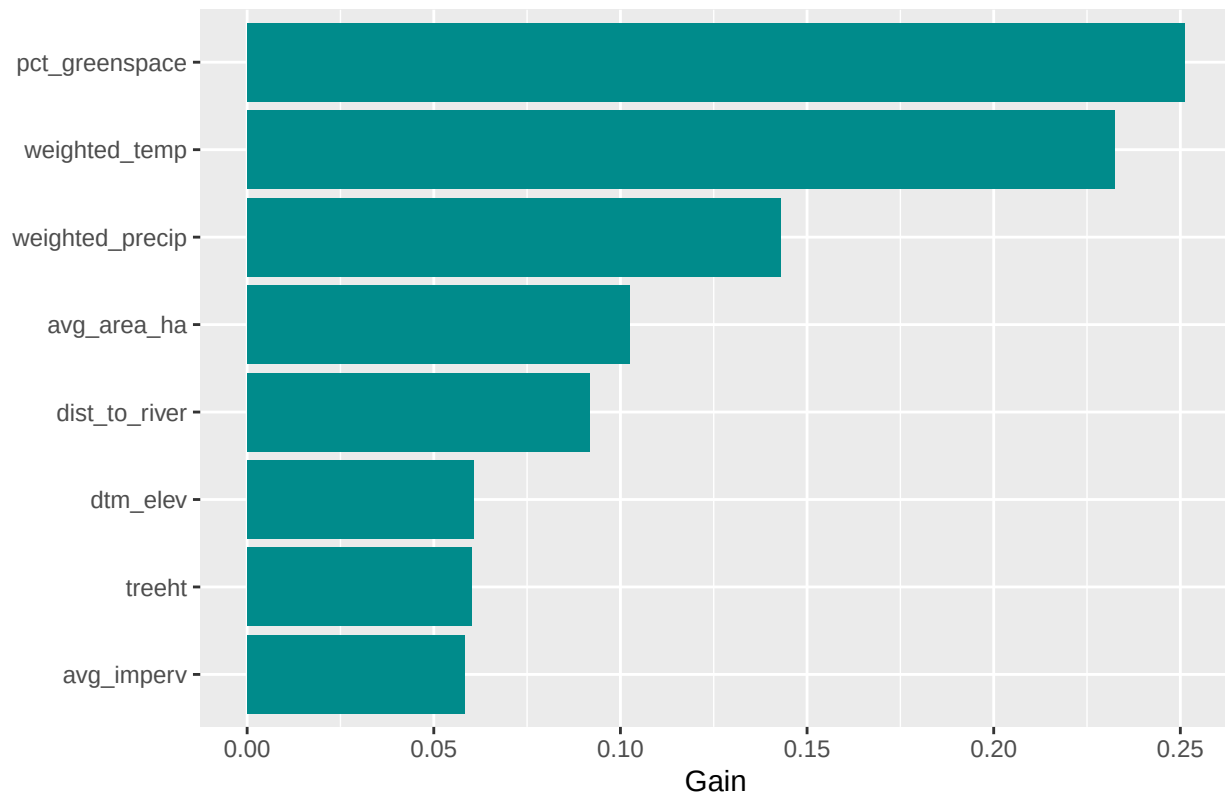
| Outcome          | Success rate AUC | Prediction rate AUC |
|------------------|------------------|---------------------|
| Species richness | 0.838            | 0.703               |

## Variable Importance

```
# Calculate and plot VI using `xgboost`
p2 <- richness_fit_xgb %>%
  extract_fit_engine() %>%
  xgboost::xgb.importance(model = .) %>%
  as.data.frame() %>%
  ggplot(aes(x = reorder(Feature, Gain), y = Gain)) +
  geom_col(fill = "cyan4") +
  coord_flip() +
  labs(title = "Variable importance: species richness classification (XGBoost)",
        x = NULL, y = "Gain")
```

p2

Variable importance: species richness classification (XGBoost)



Compared to XGBoost modelling with flood risk factors, this model is essentially unchanged when removing RoFRS and RoFSW. There is a 0.03 increase in success rate AUC (0.835 to 0.838) and 0.03 decrease in prediction rate AUC (0.706 to 0.703) in the model without flood risk. Variable importance is largely similar. It is possible that the flood risk indicators are redundant dependent on the other predictors, i.e. the other predictors already capture what the flood risk indicators do.

#SHAP

## Random Forest

```
# Extract model & set predictors
rf_model <- richness_fit_class %>%
  extract_fit_engine()
predictors <- df %>%
  dplyr::select(all_of(predictors))

# Set probability of "high"
predictor_function <- function(object, newdata) {
  predict(object, data = newdata)$predictions[, "high"]}

# Calc SHAP values
set.seed(399)

rf_shap <- kernelshap(
  object = rf_model,
```

```
X = predictors,
pred_fun = predictor_function)
```

```
## Sampling 200 rows from X as background data.
```

```
## Exact Kernel SHAP values
```

```
## |
```

```
# Plot
```

```
sv_rf <- shapviz(rf_shap)
```

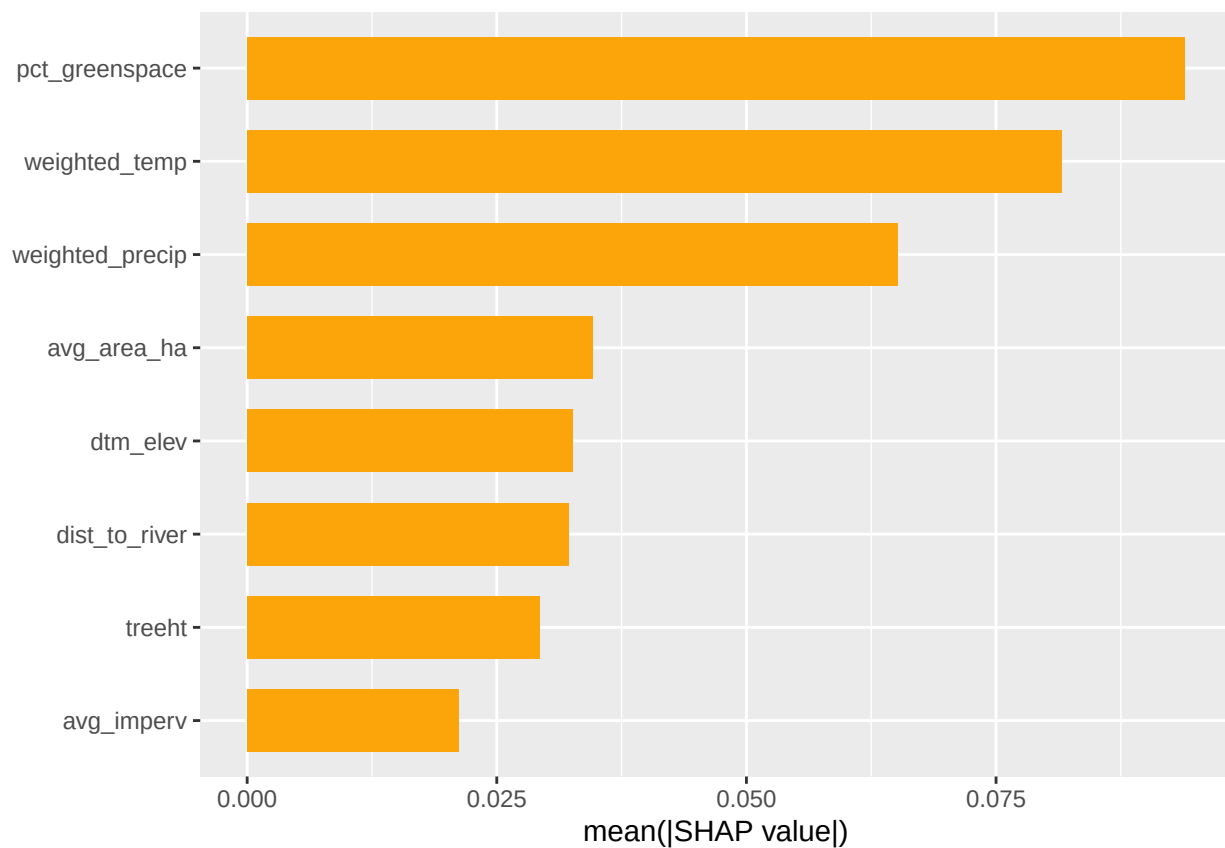
```
p1 <- sv_importance(sv_rf)
```

```
p2 <- sv_importance(sv_rf, kind = "beeswarm")
```

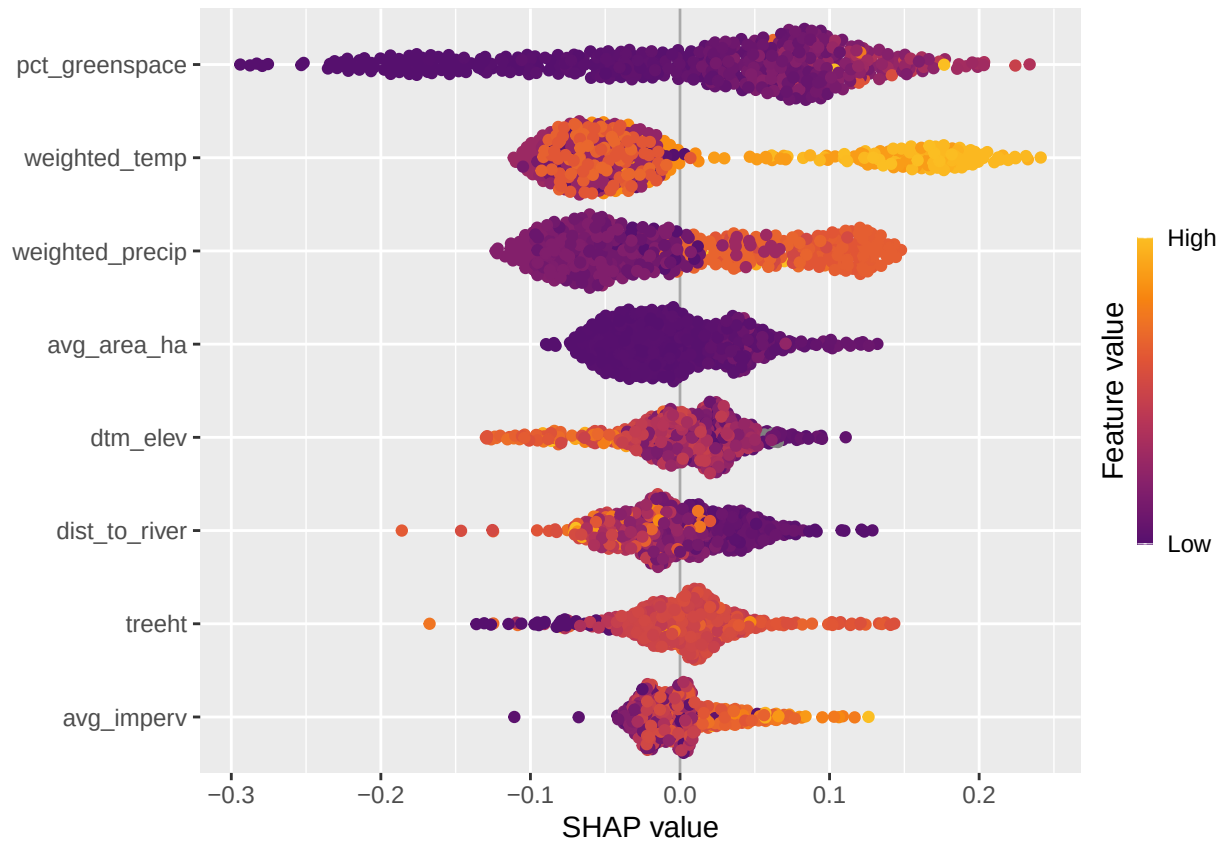
```
rfcombined <- p1 + p2
```

```
#ggsave("rf_shap.png", rfcombined, width = 12, height = 5, dpi = 300)
```

```
p1
```



```
p2
```



## XGBoost

```
#Extract the fitted model
xgb_model <- richness_fit_xgb %>%
  extract_fit_engine()

#Extract the recipe
xgb_recipe <- richness_fit_xgb %>%
  extract_recipe()

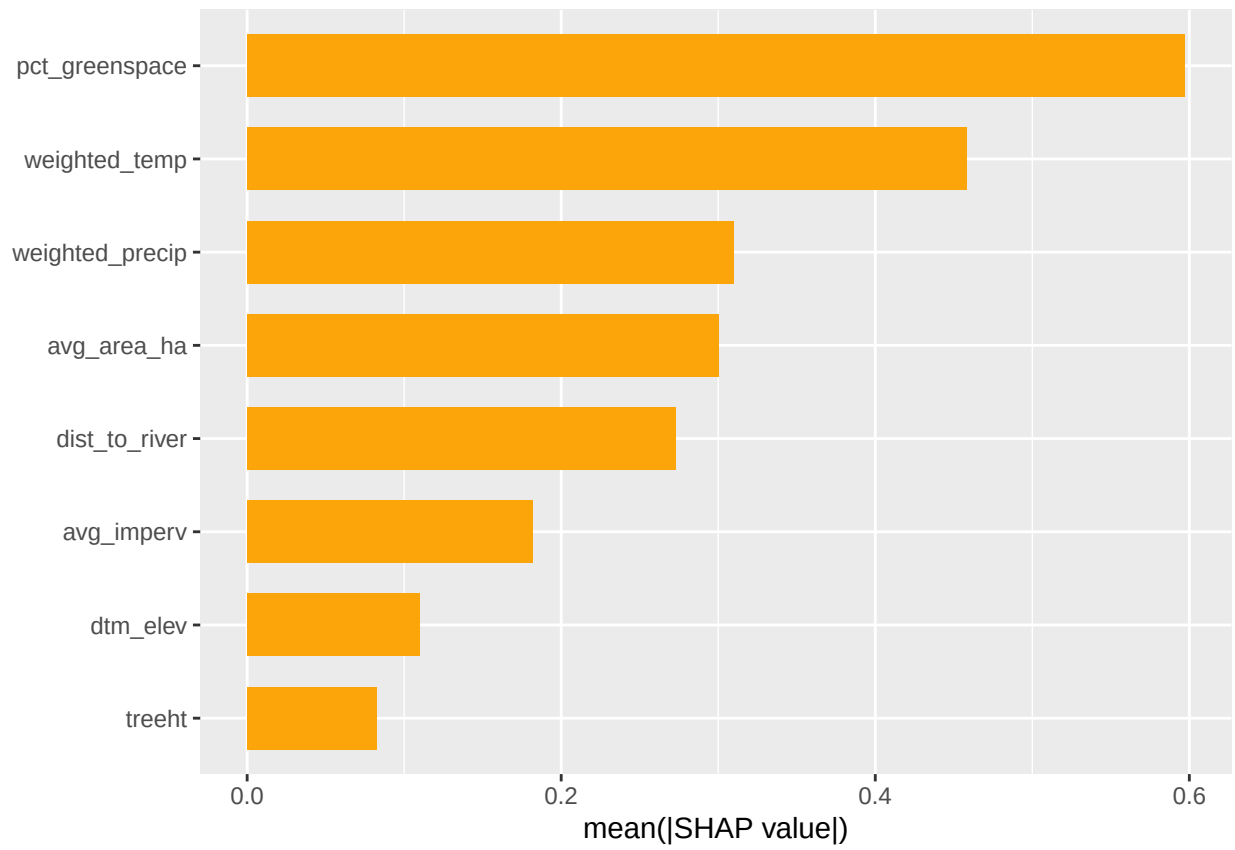
#Bake predictors as before, for continuity & comparability
x_baked <- bake(xgb_recipe, new_data = df, all_predictors())

#Calculate treeSHAP values
sv_xgb <- shapviz(xgb_model, X_pred = as.matrix(x_baked), x = x_baked)

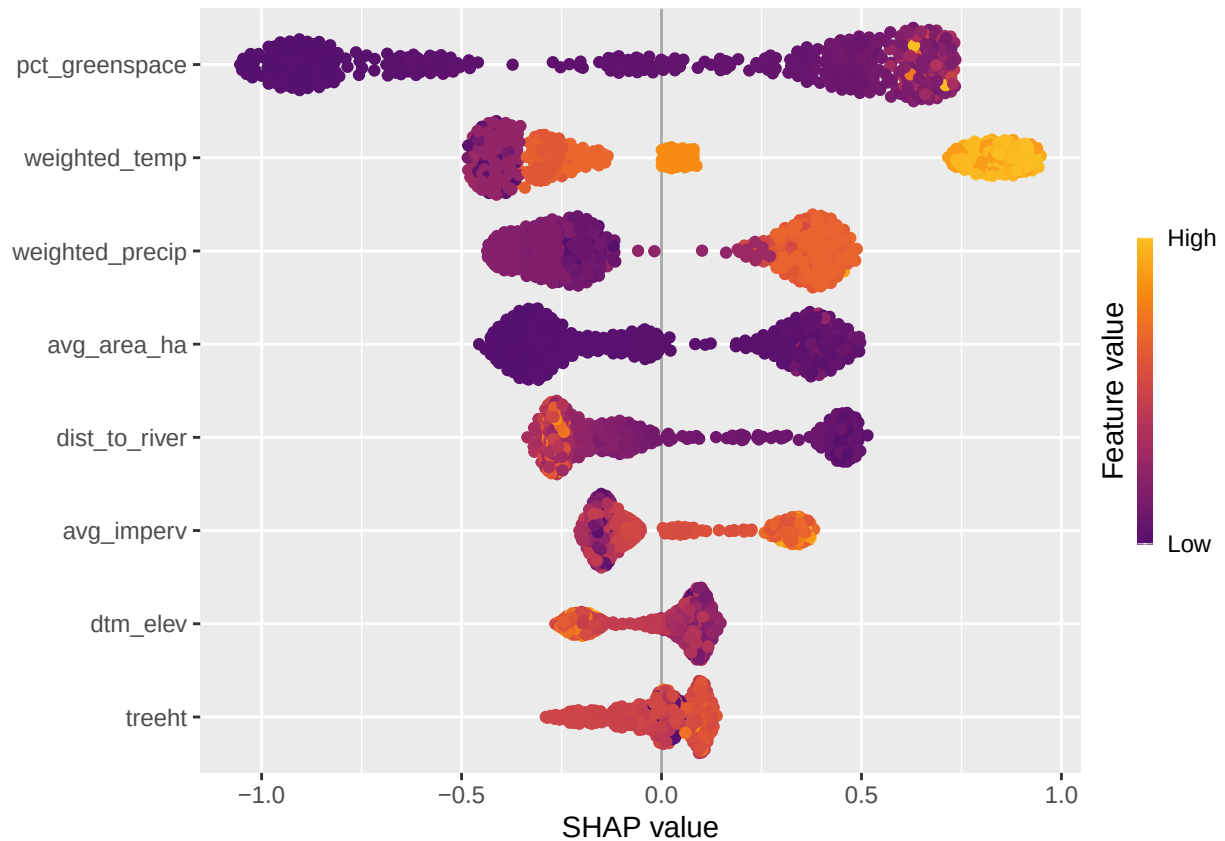
#Flip - model currently predicting "low" richness. Want "high" for comparison
sv_xgb$$S <- -sv_xgb$$S
sv_xgb$baseline <- -sv_xgb$baseline

#Plots
p3 <- sv_importance(sv_xgb)
p4 <- sv_importance(sv_xgb, kind = "beeswarm")
```

p3



p4



Overall, removing RoFRS and RoFSW had little effect on model performance or importance and direction of the remaining predictors. The small changes in AUC scores provide limited predictive information for species richness within the models. This is consistent with the previous SHAP results where RoFRS and RoFSW had relatively low contributions. Their limited contribution may reflect overlap with information already captured by other variables, rather than an absence of any ecological relationship between flood risk and species richness.